3.1 — Diferencia entre `$queryRaw` y `$executeRaw`

Ambos métodos permiten ejecutar SQL escrito a mano directamente contra la base de datos, "saltándose" el Prisma Client. Son el modo "híbrido" de Prisma: seguís usando la conexión administrada por Prisma, pero escribís SQL puro.

`$queryRaw` — para consultas que devuelven filas

```
const resultado = await prisma.$queryRaw<{ nombre: string; total: number }[]>`
    SELECT nombre, SUM(total) AS total
    FROM pedidos
    GROUP BY nombre
`;
// resultado es un array de objetos tipados
```

- **Retorna:** `Promise<T[]>` — un array de objetos, uno por cada fila devuelta por la base de datos.
- **Se usa para:** `SELECT`, o cualquier sentencia que produzca un resultado que necesitás leer en el código (informes, estadísticas complejas, consultas con múltiples JOINs, subconsultas, funciones de ventana, etc.).
- **Cuándo conviene:** cuando Prisma Client no puede expresar la consulta de forma sencilla o eficiente.

`$executeRaw` — para sentencias que modifican datos

```
const filasAfectadas = await prisma.$executeRaw`
    UPDATE productos SET stock = stock - ${cantidad} WHERE id_producto = ${id}
`;
// filasAfectadas es un número entero
```

- **Retorna:** `Promise<number>` — la cantidad de filas afectadas (equivalente a `affectedRows` en mysql2).
- **Se usa para:** `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, o cualquier sentencia DML que no devuelve filas.
- **Cuándo conviene:** cuando necesitás una operación masiva o condicional en SQL que sería complicada de armar con `update()` / `updateMany()` de Prisma Client.

Característica	$queryRaw	$executeRaw
Tipo de SQL	SELECT (lectura)	INSERT / UPDATE / DELETE (escritura)
Valor que retorna	T[] (array de filas)	number (filas afectadas)
Uso típico	Informes, estadísticas, JOINs complejos	Actualizaciones masivas, borrados condicionales

3.2 — Tagged Template Literals y protección contra SQL Injection

¿Qué son los tagged template literals?

Cuando escribimos:

```
prisma.$queryRaw`SELECT * FROM clientes WHERE dni = ${dni}`
```

No es una template string común. Es una **tagged template literal**: JavaScript llama a la función `$queryRaw` pasándole **dos argumentos separados**:

1. `['SELECT * FROM clientes WHERE dni = ', '']` — el array de las partes fijas del SQL.
2. `[dni]` — el array de los valores interpolados.

Prisma recibe esos argumentos por separado y nunca los concatena. En cambio, arma una consulta parametrizada que envía a MySQL así:

```
Consulta: SELECT * FROM clientes WHERE dni = ?
Parámetro: [valor_de_dni]
```

MySQL trata el parámetro como **dato puro**, nunca como código SQL. El valor puede contener comillas, punto y coma, o cualquier carácter especial y la base de datos lo interpreta literalmente como texto, sin ejecutarlo.

¿Por qué es peligroso concatenar strings?

Si en cambio escribiéramos:

```
// ⚠ PELIGROSO — NO HACER ESTO
const sql = `SELECT * FROM clientes WHERE dni = '${dni}'`;
const resultado = await prisma.$queryRawUnsafe(sql);
```

Un atacante podría enviar como DNI el valor:

```
' OR '1'='1
```

Y la consulta quedaría:

```
SELECT * FROM clientes WHERE dni = '' OR '1'='1'
```

Que devuelve **todos los clientes** sin importar el DNI. Con variantes más agresivas podría borrar tablas (`DROP TABLE`), extraer datos de otras tablas, o escalar privilegios.

Conclusión

El tagged template literal es el mecanismo de seguridad que hace que `$queryRaw` sea seguro: los valores de las variables nunca se mezclan con el SQL. Por eso `$queryRawUnsafe` (que sí concatena) es inseguro y solo debe usarse cuando los valores vienen de fuentes completamente controladas por el programador (nunca del usuario).

3.3 — ¿Cuándo justificarías abandonar el Prisma Client y usar SQL puro?

El Prisma Client cubre la enorme mayoría de los casos del día a día. Sin embargo, hay situaciones específicas donde escribir SQL puro es la decisión correcta:

1. Consultas con lógica compleja no soportada por Prisma Client

Prisma no tiene soporte nativo para **funciones de ventana** (`ROW_NUMBER()`, `RANK()`, `LEAD()`, `LAG()`), **CTEs recursivas** (`WITH RECURSIVE`) ni algunos tipos de pivots o `CASE` complejos. Si la consulta lo requiere, SQL puro es el camino.

Ejemplo: ranking de clientes por ventas en un periodo específico con `RANK() OVER (PARTITION BY ...)`.

2. Informes o dashboards con múltiples JOINs y agregaciones en una sola pasada

Cuando necesitás cruzar muchas tablas y hacer varios `SUM`, `COUNT`, `AVG` al mismo tiempo, armar eso con Prisma Client implica múltiples llamadas a la BD y luego combinar los resultados en JavaScript. Una sola query SQL lo hace en un solo viaje, más eficientemente.

Ejemplo: el informe de la consigna 3.4 de este proyecto.

3. Operaciones masivas de actualización o borrado condicional

Prisma `updateMany()` es útil, pero no soporta actualizaciones que referencia a la misma fila (como `SET stock = stock - cantidad`). Tampoco soporta actualizaciones que dependen de un JOIN (`UPDATE productos p JOIN proveedores pv ON ... SET p.precio = ...`).

Ejemplo: descuento masivo de stock después de procesar un pedido.

4. Uso de características específicas del motor de BD

Si necesitás funciones propias de MySQL que Prisma no expone (como `JSON_EXTRACT`, `ST_Distance` para geolocalización, o `FULLTEXT` search), tenés que bajar a SQL puro.

5. Performance crítica con consultas muy lentas

En raras ocasiones, la query que genera Prisma Client puede ser sub-óptima para un caso concreto. Si el profiler de la BD muestra que la query generada es ineficiente y reescribirla en SQL la acelera significativamente, tiene sentido usar `$queryRaw`.

Regla general

Usar Prisma Client como primera opción siempre. Recurrir a `$queryRaw` / `$executeRaw` solo cuando la consulta no se puede expresar bien con el ORM, cuando la complejidad de la lógica lo justifica, o cuando hay un problema de performance demostrable.